

COSCON 2026 • TAIPEI

Containarium

An SSH-native agent runtime, built because our hardware was sitting idle.

HsinHo Yeh — Founder of FootprintAI, OSS Maintainer
github.com/FootprintAI/Containarium • Apache 2.0



Who is talking

Computer Science & Information Engineering, National Cheng Kung University



01

Academia Sinica

Research on computer vision, pattern recognition, and high performance data mining.

02

HTC

Cloud infrastructure. Fleets of machines, and the bills that come with them.

03

Igloo

First employee at the SEA insurtech unicorn. Building dev platforms under startup constraints.

04

Footprint AI

Founder. Containarium came out of our own infrastructure bill.

05

CCU & NYUST

Mentor on open-source projects with student teams at both universities. Recent contributions to kubernetes, kubeflow, llama.cpp and whisper.cpp.

What this talk covers

- 01 Where the utilisation goes on AI and VM fleets
- 02 Version zero: one VM per developer, and what it cost
- 03 LXC, sshpiper and persistent disk: the design of Containarium
- 04 Agent orchestration in production, the Kubernetes backend, and the community
- 05 User stories: dev boxes, QA triage, and overnight GPU benchmarks

THE PROBLEM WE WORK ON

Hardware utilisation across AI data centres and VM fleets

The machine is bought and paid for around the clock. The work on it is bursty, interactive, and mostly asleep.

STACK 01

Inference accelerator on top of vLLM

Squeezing more tokens out of the GPUs you already own.

STACK 02

Containarium, a lightweight box

Open source. The subject of the rest of this talk.

Where the capacity actually goes

8%

Average CPU across a per-developer VM over a working week

3 hrs

Typical daily window of real interactive use per box

100%

Of the hours you are billed for, whether or not anyone is logged in

Agents make this worse, not better. An agent holds a box for days while doing minutes of work in it.

One VM per developer, reached over SSH

```
dev-01  7
dev-02  |
dev-03  |    4 vCPU / 32 GB each
dev-04  |—— public IP, own SSH daemon
...     |    own apt cache, own everything
dev-10  J
```

```
$ ssh dev-04
Welcome to Ubuntu 24.04 LTS
```

It shipped in an afternoon and everybody was happy. SSH is the API developers already know.

Then the fleet grew, every box stayed on around the clock, and the invoice stopped being a rounding error.

What ten idle boxes cost per month

LINE ITEM	UNIT	10 DEVELOPERS
Compute — 4 vCPU, 32 GB, on demand	\$188	\$1,880
Boot + work disk — 100 GB SSD	\$17	\$170
Static IP and egress, rounded	\$8	\$80
Monthly total	—	\$2,130

Estimate at public list pricing, single region, 730 hours. Roughly \$25k a year to keep ten mostly-idle Linux machines warm.

The same ten people on one host

BEFORE

10 × VM, 4 vCPU / 32 GB

\$2,130 / mo

40 vCPU and 320 GB allocated. Around 8% of it used.

AFTER

1 × host, 16 vCPU / 192 GB, spot

\$455 / mo

Includes 1 TB of ZFS-backed persistent storage for all tenants.

Memory is the binding constraint, not cores, so we bought memory. The open question was how to slice one host into ten machines that still feel like machines.

LXC is a system container

Same kernel namespaces and cgroups as Docker. Different contract: instead of one process, the container boots a full Linux userland.

Real `systemd`, real users, real `sudo`, real package manager, a writable filesystem that survives reboots. It feels like a VM and costs like a process.

We run Incus, the modern LXC manager, with unprivileged containers: container root is not host root.

```
$ ssh alice
alice@alice-container:~$ systemctl status
    State: running
    Jobs: 0 queued

alice@alice-container:~$ sudo apt install postgresql
alice@alice-container:~$ docker run hello-world
Hello from Docker!
```

LXC next to the alternatives

	LXC / INCUS	DOCKER	VM / VIRTUALBOX
Unit	Whole OS	One process	Whole machine
Kernel	Shared with host	Shared with host	Its own
init / systemd	Yes	No, by design	Yes
Start time	A few seconds	Sub-second	Tens of seconds
Memory overhead	Tens of MB	Near zero	GB per guest
Fits an agent that installs things	Yes	Awkward	Yes, expensively

The four questions we always get

Why not Docker?

Docker packages an application. We hand over a machine. Docker runs fine inside a Containarium box; the reverse is not really a thing.

Why not Vagrant or VirtualBox?

Those orchestrate guests on a laptop. We are consolidating shared remote hardware that many people and agents reach at once.

Why not Kubernetes?

Kubernetes is the layer underneath. Containarium runs on top of it, and it is a supported backend since June.

Why not Codespaces or Gitpod?

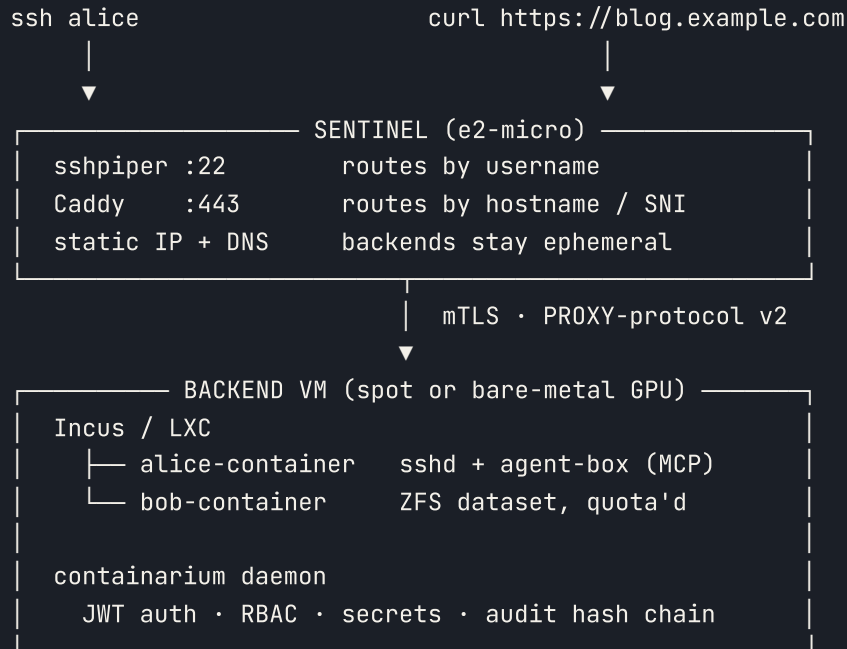
Those are IDEs with a per-hour meter and an auto-delete policy. Our boxes are IDE-agnostic, persistent, and self-hosted.

VERSION ONE

LXC + SSH reverse proxy + persistent disk

Three parts, none of them new. The design work was in the contract between them: `ssh -a` reaches Alice's box, and nothing else.

How a connection lands



sshpipec is the whole trick. It terminates SSH on the sentinel and routes by username, so tenants never share a daemon and fail2ban bans are per user.

The sentinel is tiny. A free-tier instance holds the static IP, so the expensive backend can be a spot VM that comes and goes.

Recovery is automatic. Preemption is detected in about ten seconds, a maintenance page is served, and the backend is back in roughly eighty-five.

Why the disk had to be persistent

Survives the host

Containers live on ZFS datasets on the backend. Spot termination and reboots do not take the tenant's work with them.

Snapshots and quotas

Daily snapshots with 30-day retention, compression, checksums, and a hard per-tenant quota so one box cannot eat the host.

Warm caches

Package caches, build artefacts and model weights stay put between sessions, which is most of the perceived speed.

Consolidation only works if nobody notices the consolidation. Persistence is the part that buys the silence.

One box, five commands

```
$ curl -fsSL ../hacks/install.sh | sudo bash    # incus + daemon, ~5 min

$ containarium create alice --ssh-key ~/.ssh/id_ed25519.pub
$ containarium ssh-config sync                  # one Include line in ~/.ssh/config
$ ssh alice                                    # through the sentinel

$ containarium expose-port alice \
    --container-port 8080 --domain blog.example.com
→ https://blog.example.com is live, TLS terminated on the sentinel
```

The CLI is canonical. REST, gRPC, the web UI and both MCP servers call the same functions underneath.

An MCP server inside every box

agent-box runs in the container and speaks MCP over stdio, wrapped by SSH on the client side. Typed tools instead of an agent guessing at a TTY.

shell_exec is bounded by a timeout and a 256 KiB output cap. File operations can be pinned to a project root with AGENTBOX_ROOT.

A second MCP server on the host covers the admin verbs: create, list, delete, expose, metrics.

```
~/.claude.json
{
  "mcpServers": {
    "containarium-box": {
      "command": "ssh",
      "args": ["alice", "agent-box"]
    }
  }
}

tools: shell_exec · read_file · write_file
      list_directory · move_file · delete_file
```

IN PRODUCTION, ON OUR OWN TEAM

Everyone runs coding agents. Almost nobody talks about scaling them.

Two things make it work: a software quality process, and one sandbox per agent. We shipped both as an agent skill plus a runtime.

One box per agent, one role per box

orchestrator

└─ pm-agent	writes specs, never touches build
└─ eng-agent-1	CLI toolchain, compilers, caches
└─ eng-agent-2	CLI toolchain, compilers, caches
└─ qa-agent	headless browser, E2E, seeded data
└─ triage-agent	per-issue stack, debug tooling
└─ bench-agent	ephemeral, spawned per sweep

6 agents · 6 independent Linux boxes · 1 host

~100

commits a day, per engineer, with PM and engineering agents never stepping on each other. Not a KPI — a side effect of scaling cleanly.

Engineers install their own toolchain. QA installs a browser and simulates environments. Nobody pollutes anybody else's box, and nothing blows up your laptop.

A QA agent that triages, not tests

01

Open a box

Every issue needs a different stack and different debug tooling. That is exactly why it gets its own box.

02

Try to reproduce

The most expensive step in the whole pipeline, and the one humans skip most often.

03

Report either way

Reproduced, with steps and environment. Or not reproduced, with the conditions it tried and a hypothesis from the codebase.

04

Hand off confirmed only

The engineering agent starts fixing only once a repro exists.

The underrated outcome is the second one. A human engineer is reluctant to say "cannot reproduce", so the issue stays open while they change things at random. The agent has no such reluctance: it lists the five environments it tried and passes the question back to the user. Bugs pile up because nobody has time to reproduce them, not because fixing is slow.

A benchmark agent that runs while we sleep

```
00:05  containarium create bench-0412 --gpu 0 --stack gpu
00:07  install harness + workload deps
00:09  sweep: batch × concurrency × quantisation
04:41  collect results, write report, post to issue
04:43  containarium delete bench-0412
      cost = 4h 38m, not 24h
```

The agent spawns its own box with a GPU passed through at the PCI level, installs the benchmarking tools, runs the experiment across parameter sets, writes up a report, and tears the box down when it is finished.

A dedicated GPU node would have sat idle for the other twenty hours. The card goes back to the pool at teardown, which is the utilisation argument from the start of the talk, closing the loop.

What bounds the blast radius

KEY	The agent holds an SSH key, not a kube-apiserver token. No path to the control plane or the host OS.	KERNEL	Unprivileged LXC with AppArmor profiles per container. Container root is not host root.
EGRESS	Per-tenant egress allowlists enforced in the kernel by TC_INGRESS eBPF programs, portable across LXC and Kubernetes.	TOKENS	JWT with issuer, audience, jti and per-tool scopes. Refresh tokens are single-use; revocation is by jti.
SECRETS	Envelope encryption through Vault Transit or GCP KMS, delivered on tmpfs so they never appear in /proc.	AUDIT	Append-only log with a SHA-256 hash chain and containerium audit verify to detect tampering.

The same box, as a pod

Shipped in June. The daemon reconciles a per-tenant namespace, StatefulSet, headless Service and default-deny NetworkPolicy, then programs the sshpiper Pipe CRD.

The pod runs with `automountServiceAccountToken: false` and satisfies the restricted Pod Security profile. The agent still gets in over SSH and still holds no cluster credential.

GPUs come through `nvidia.com/gpu` limits. An agent cannot tell which substrate it landed on.

```
# LXC / Incus, the default
$ containerium daemon

# the cluster you already operate
$ containerium daemon --runtime=k8s
```

Both backends share one CLI, one MCP surface, one auth model. Local bring-up on kind takes under five minutes.

Where this sits among the neighbours

[kubernetes-sigs/agent-sandbox](#)

Exec-based, in-cluster

Reached via `kubectl exec`, which means the agent holds cluster credentials. NetworkPolicy rather than per-tenant eBPF. We are talking with them.

[NVIDIA/OpenShell](#)

eBPF, NVIDIA stack

Strong kernel-level controls, tied to the NVIDIA stack and a custom client rather than a portable, MCP-native surface.

[e2b](#) · [Modal](#) · [Daytona](#)

Hosted and ephemeral

The closest peers in intent. SaaS-only and process-per-call shaped. Pick them if hosted and short-lived is what you want.

Containerium's slot: self-hosted, persistent, SSH-native, Apache 2.0, and the same contract whether the substrate is LXC or your cluster.

Where the project is today

1,013

commits on main

117

releases

Apache 2.0 **Prod**

no CLA, PRs triaged weekly

GCP multi-region and bare-metal GPU nodes

SHIPPED Q2

Kubernetes backend, agent-box MCP, ssh-config and expose-port CLI

Q3 2026

agent-box tier two: MCP Roots and background process management

Q4 2026

OSS v1.0 cut: stable API surface and contribution guide

FIVE MINUTES ON A FRESH UBUNTU VM

Take one box for a walk

```
curl -fsSL \  
  https://raw.githubusercontent.com/footprintai/containerium/main/hacks/install.sh \  
  | sudo bash  
sudo containerium create alice --ssh-key ~/.ssh/id_ed25519.pub  
ssh alice
```



Scan or click for the repo

github.com/FootprintAI/Containerium

containerium.dev

github.com/FootprintAI/Containerium

55s demo: youtu.be/IBDDD_tb8FY

COSCU 2026 · CONTAINERIUM